



基于深度学习的商品识别系统（网页版+YOLOv8_v7_v6_v5代码+训练数据集）



逗逗班学Python

编辑于 2024年07月26日 14:48



收录于文集

网页版-深度学习检测系统 · 80篇



摘要：在本博客中，我们深入探讨了**基于YOLOv8/v7/v6/v5的商品识别系统**，其中核心技术基于**YOLOv8**，同时整合了YOLOv7、YOLOv6、YOLOv5的算法进行了细致的性能指标对比分析。我们全面回顾了国内外在商品识别领域的研究进展，详细介绍了**数据集的处理方法、YOLO算法的核心原理、模型构建过程以及训练策略**。特别地，本文还展示了如何设计一个基于**Streamlit的交互式Web应用界面**，该界面不仅支持图像和视频识别，还可以接入实时摄像头进行商品识别。用户可以轻松上传不同的训练模型（包括YOLOv8、v7、v6、v5）来进行推理预测，界面的友好性使得参数调整变得简单直观。本博客附带了**完整的Web界面设计方案、深度学习模型的实现代码以及训练所需数据集的下载链接**，为广大研究者和开发者提供了一套完整的商品识别系统解决方案。

👉点击跳转至文末所有涉及的**完整代码文件**下载页👈

完整资源中包含数据集及训练代码，环境配置与界面中文字、图片、logo等的修改方法请见视频，项目完整文件下载请见演示与介绍视频的简介处给出：



演示与介绍视频：[🔗 网页链接](#)

YOLOv8/v7/v6/v5项目合集下载：<https://mbd.pub/o/bread/mbd-ZZ2Zm51u>

YOLOv8和v5项目完整资源下载：<https://mbd.pub/o/bread/mbd-ZZ2Zmp9y>

YOLOv7项目完整资源下载：<https://mbd.pub/o/bread/mbd-ZZ2Zm51r>

YOLOv6项目完整资源下载：<https://mbd.pub/o/bread/mbd-ZZ2Zm51p>

若您想获得博文中涉及的实现完整全部资源文件（包括测试图片、视频，py, UI文件，训练数据集、训练代码、界面代码等），这里已打包上传至博主的面包多平台，见可参考博客与视频，已将所有涉及的文件同时打包到里面，点击即可运行，完整文件截图如下：



目录



1



0



0



0



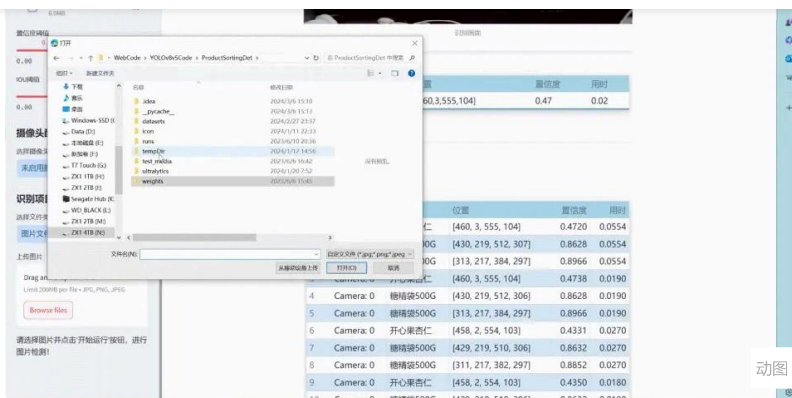
0



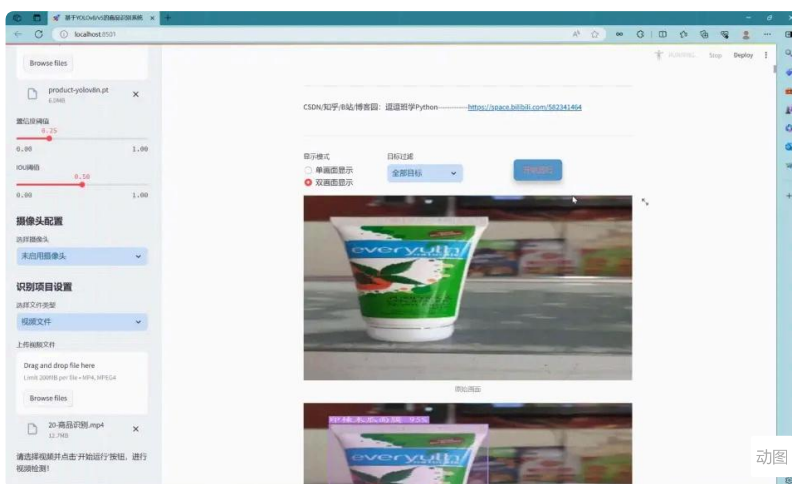
首页 番剧 直播 游戏中心 会员购 漫画 赛事



登录



(3) 选择视频文件检测：系统支持用户上传视频文件进行商品识别。上传的视频将被系统逐帧分析，以识别和标记视频中每一帧的商品。用户可以观看带有商品识别标记的视频，了解视频中商品的变化。



(4) 选择不同训练好的模型文件：系统集成了多个版本的YOLO模型（如YOLOv8/v7/v6/v5），用户可以根据自己的需求选择不同的模型进行商品识别。这一功能使得用户能够灵活地比较不同模型的表现，以选择最适合当前任务的模型。



回到旧版



的总体设计，强调了其用户友好性和直观性，使用户能够轻松地进行操作和管理。

实时摄像头检测功能允许用户开启摄像头，实时捕捉视频流并在视频中标记出戴商品的人员，展示了算法在实时环境下的应用效率和准确性。通过**图片和视频文件检测**功能，用户可以上传图片或视频文件，系统将自动进行商品识别，并显示检测结果，支持多种文件格式和大小。

系统的核心之一是其**模型选择功能**，用户可以根据需要选择不同的训练模型（YOLOv8/v7/v6/v5）进行检测，以获得最佳的检测效果。同时，**显示设置**允许用户根据需求选择检测画面和原始画面的显示方式，并通过下拉框单独标记和显示特定目标的检测结果。

在管理和优化检测结果方面，我们提供了强大的工具。**检测结果管理**功能使检测结果以表格形式直观展示，用户可以动态调整检测算法的置信度阈值和IOU阈值，以优化检测效果。为了方便后续分析和报告制作，**结果导出功能**支持用户将检测结果和标记的图像或视频导出为csv或avi格式文件。

此外，我们还重点关注了用户交互和体验的优化，通过提供直观的界面布局、快速响应时间和易于导航的界面，确保了用户可以高效地使用所有功能。通过一系列使用示例，我们展示了如何利用这些功能进行有效的商品识别和分析，无论是在实时监控还是事后分析中，都能满足用户的需求。

2. 绪论

2.1 研究背景及意义

在当今快速发展的零售行业和电子商务领域，商品识别技术的重要性日益凸显。随着智能零售和无人超市的兴起，对高效、准确的商品识别系统的需求急剧增加。商品识别不仅能够提高库存管理的效率，减少人工成本，还能为顾客提供更加个性化的购物体验。此外，在线下零售中，商品识别技术可以实现快速结账，优化顾客购物流程，增强顾客满意度。

随着深度学习技术的飞速发展，尤其是卷积神经网络（CNN）在图像识别领域的广泛应用，使得商品识别的准确度和效率得到了显著提升。YOLO（You Only Look Once）系列算法作为图像识别领域的先驱之一，以其高效的识别速度和良好的准确性，成为商品识别系统的理想选择。从YOLOv1到最新的YOLOv8，每一次迭代更新都在性能和准确性上有所突破，特别是在处理复杂背景和遮挡条件下商品的识别能力上有显著提升。

然而，尽管现有的YOLO系列模型已经能够在多个场景下实现较为准确的商品识别，但仍然面临着一些挑战。例如，商品的多样性和复杂性要求模型具有更高的泛化能力；实时识别场景下对于识别速度的要求更为严格；此外，模型的轻量化和优化也是使得商品识别技术能够广泛应用于移动设备和边缘计算设备的关键。

针对以上挑战，本博客将深入探讨基于YOLOv8/v7/v6/v5的商品识别系统的研究背景及其在当今社会的重要意义，分析国内外在此领域的研究现状和技术进展。我们将重点探讨目前商品识别面临的关键问题，以及通过最新的YOLO算法及其它深度学习技术改进，结合最新的数据集和性能优化策略，来解决这些问题的可能方案。本博客的主要贡献在于提供了一种集成了最新YOLO算法改进、



目录



1



0



0



0



0

2.2 国内外研究现状

在目标检测领域，尤其是针对商品识别系统的研究，近年来的技术进步显著，新的算法和方法不断涌现，使得商品检测的准确性和效率得到了大幅提升。从传统的基于特征的方法到现代的基于深度学习的算法，目标检测技术经历了翻天覆地的变化。特别是YOLO（You Only Look Once）系列算法的快速发展，为实时商品检测提供了强有力的技术支持。



YOLO系列算法自从2016年首次提出以来，以其快速和准确的检测性能受到广泛关注。YOLOv1的创新之处在于将目标检测问题转化为单一的回归问题，通过一个深度网络直接从图像像素预测边界框和类别概率。随后，YOLOv2、YOLOv3对网络结构进行了改进，引入了Batch Normalization、多尺度训练等技术，进一步提高了检测的准确性和速度。到了YOLOv4，作者通过大量实验验证了各种技术和策略的有效性，并集成了CSPNet、Mish激活函数、自适应锚框策略等创新点，使得YOLOv4在保持高速度的同时，准确率也得到了显著提升[1]。

近年来，Transformer架构因其在自然语言处理领域的巨大成功而引起了图像处理领域的广泛关注。Vision Transformer（ViT）是将Transformer应用于图像分类任务的开创性工作，通过将图像切割成多个小片段（patches），然后将这些片段视为序列数据输入到标准的Transformer模型中，ViT展示出了与卷积神经网络相媲美甚至更优的性能[2]。此后，DETR（Detection Transformer）将Transformer引入到目标检测领域，通过端到端的方式直接预测对象的边界框和类别，简化了传统目标检测流程，避免了复杂的后处理步骤，显著提高了检测任务的效率和准确性[3]。

除了上述算法外，RetinaNet通过引入Focal Loss来解决类别不平衡的问题，显著提升了小目标检测的性能。Focal Loss设计用来减轻简单样本在训练过



目录



1



0



0



0



0



了丰富的模型库和工具，支持快速实验和研究，成为目标检测领域研究者和工程师的重要资源[5]。

随着技术的不断进步，目标检测算法在商品识别领域的应用也越来越广泛。然而，实际应用中还面临着多种挑战，如不同环境下的光照变化、商品之间的遮挡、相似商品的区分等。因此，未来的研究需要不断探索更高效、更准确的算法，并结合领域知识来克服这些挑战。

通过深入分析当前的研究现状，我们可以看到目标检测技术在商品识别系统中的广泛应用和重要意义。随着新算法的不断提出和旧算法的不断优化，商品识别的准确性和效率将进一步提升，对促进智能零售发展具有重要作用。

2.3 要解决的问题及其方案

2.3.1 要解决的问题

商品识别的准确性和速度

在零售业中，准确快速地识别商品是至关重要的。由于商品种类繁多，且每种商品的外观、尺寸和包装可能有着显著的差异，系统需要能够准确地识别并区分各种商品。此外，考虑到实际应用场景中可能需要处理大量商品，识别速度也是一个关键因素。因此，开发一个既准确又快速的商品识别模型是本项目的核心任务。

环境适应性和模型泛化能力

零售环境的多样性（如不同的照明条件、复杂的背景、商品排列方式等）对识别系统提出了额外的挑战。系统需要具备良好的环境适应性和模型泛化能力，以确保在各种零售环境中都能保持高识别准确率。

用户交互界面的直观性和功能性

为了确保用户（零售商、库存管理人员等）能够有效利用商品识别系统，需要开发一个直观、易用且功能丰富的用户界面。界面设计应便于用户进行常规操作，如上传商品图像/视频、实时商品检测、切换不同的模型文件等。

数据处理能力和存储效率

商品识别系统将处理大量的图像和视频数据，因此需要具备强大的数据处理能力和高效的存储机制。这不仅关系到识别的实时性，还涉及到数据的长期管理和查询效率。

系统的可扩展性和维护性

考虑到零售业的快速发展和技术的不断进步，系统需要设计得足够灵活，以便未来可以轻松集成新的模型或功能。同时，系统的维护性也是关键，确保能够持续稳定运行，并及时进行更新和升级。

2.3.2 解决方案

针对基于YOLOv8/v7/v6/v5的商品识别系统面临的挑战，我们提出了一套综合的解决方案，旨在优化模型性能，提高用户交互体验，并确保系统的高效运行和易用性。以下是我们拟采用的具体方法：

1. 深度学习模型的选择和优化



目录



1



0



0



0



0



的商品识别任务。同时，系统也支持YOLOv7、v6、v5等其他版本，为不同的应用场景和性能需求提供灵活选择。

数据增强：为了增强模型的泛化能力，将采用多样的数据增强技术，包括但不限于随机裁剪、缩放、旋转和色彩调整等，以模拟多变的零售环境中的商品图像。

迁移学习：利用在大规模数据集上预训练的YOLO模型作为基础，通过迁移学习技术进行微调，使用特定的商品数据集进行训练，以加快模型训练速度并提升识别准确性。

2. 技术框架和开发工具

PyTorch框架：选用PyTorch作为深度学习框架，借助其灵活性和强大的GPU加速能力，快速开发和迭代模型。PyTorch的广泛社区支持和丰富的库资源，使得模型开发和优化更加高效。

Streamlit Web应用：基于Streamlit构建用户交互界面，其简洁高效的编程模型能够快速搭建交互式Web应用。用户可以通过Web界面上传图片、视频，甚至接入实时摄像头进行商品识别。

CSS美化：使用CSS对Streamlit应用进行美化，提升用户界面的视觉效果和用户体验。通过定制化的样式和布局，使得应用界面更加专业和友好。

PyCharm IDE：采用PyCharm作为开发环境，它提供了强大的代码编辑、调试和项目管理功能，帮助开发者提高开发效率和代码质量。

3. 功能实现和系统设计

多输入源支持：设计系统以支持多种输入源，包括图像文件、视频流和实时摄像头捕获，满足不同零售场景下的商品识别需求。

模型切换功能：实现动态模型切换功能，允许用户根据具体需求（如更高的准确度或更快的识别速度）选择不同的预训练模型，增加系统的灵活性和实用性。

4. 数据处理和存储策略

高效数据处理：利用PyTorch的高效数据加载和预处理机制，确保数据处理流程的高速和稳定，保障实时商品识别的需求。

智能数据存储：设计合理的数据存储方案，高效组织和索引识别结果及历史数据，支持快速查询和数据分析，同时确保数据的安全性和隐私保护。

5. 性能优化和系统测试

性能调优：定期对模型和系统进行性能分析，通过模型压缩、算法优化等手段，提升系统的运行效率和识别性能。

全面系统测试：开展全方位的系统测试，包括功能测试、性能测试和用户体验测试，确保系统的稳定性、可靠性和易用性，满足商业部署的要求。

通过实施上述方法，我们期望构建一个既准确又高效的商品识别系统，不仅能满足零售业的需求，还能提供卓越的用户体验和数据分析能力。

2.4 博文贡献与组织结构



目录



1



0



0



0



0



Streamlit框架设计的用户友好的Web界面。本研究的主要贡献可以概括为以下几点：

- 1. 深入的文献综述：**本文提供了关于商品识别领域的深入文献综述，涵盖了最新的算法发展动态，特别是对YOLO系列算法的详细分析和对比，为研究人员和开发者提供了宝贵的学术资源和技术参考。
- 2. 综合的数据集处理方法：**详细介绍了在商品识别任务中使用的数据集处理方法，包括数据增强、标注规范和预处理技术，为如何处理复杂多变的零售环境下的商品图像提供了实用的指导。
- 3. 算法选择与优化：**深入探讨了YOLOv8及其前几代版本（v7/v6/v5）在商品识别任务中的应用，包括算法的架构选择、性能优化和定制化改进，为构建高效准确的商品识别模型提供了策略和思路。
- 4. 友好的用户交互界面设计：**基于Streamlit框架，设计并实现了一个美观、易用的Web应用界面，使非技术用户也能轻松地进行商品图像的上传、检测和模型选择等操作，极大地提高了系统的可用性。
- 5. 系统性能的全面对比分析：**通过对YOLOv7/v6/v5等不同版本的算法在同一数据集上的表现进行对比分析，为选择合适的商品识别模型提供了实证依据。
- 6. 资源共享：**提供了完整的数据集和代码资源包，包括预训练模型、训练和预测的代码，使研究人员和开发者能够复现实验结果，促进了知识的共享和技术的快速迭代。



目录



1



0



0



0



0

博客后续章节的组织结构

本文接下来的章节将按照以下结构组织，以便更系统地呈现我们的研究工作和实现成果：

绪论：介绍商品识别系统的研究背景、意义和研究现状，明确本研究的目的和主要贡献。

算法原理：详细阐述YOLOv8/v7/v6/v5等深度学习算法的原理、架构特点以及在商品识别中的应用和优化策略。

数据集处理：介绍数据集的选择、预处理方法、数据增强技术和标注规范，确保数据质量和模型训练的有效性。

代码介绍：详细展示模型训练与预测的实现代码，包括PyTorch框架下的模型定义、训练循环和预测逻辑。

实验结果与分析：报告不同模型在商品识别任务上的性能表现，包括准确率、检测速度等关键指标的对比分析。

系统设计与实现：详细介绍基于Streamlit的Web应用设计和实现过程，包括用户界面设计、功能实现和用户交互逻辑。

结论与未来工作：总结本文的主要发现和贡献，讨论研究的局限性，并提出未来工作的可能方向。

通过这一系列的组织结构，我们希望读者能够全面地理解基于YOLOv8/v7/v6/v5的商品识别系统的设计理念、技术实现和应用价值。

[回到旧版](#)

3. 数据集处理



的图像，覆盖了多个商品类别和品牌。在本数据集中，我们将每个商品都与其对应的中文名称相匹配，涉及的品牌和商品范围广泛，从各种口味的Complan奶粉、Everyuth护肤品到不同包装和类型的Glucon D饮料粉，以及Nutralite的各式调味蛋黄酱等，体现了零售环境中商品多样性的真实情况。博主使用的类别如下：

```
Chinese_name = {"Complan Classic Creme": "经典奶油",
                "Complan Kesar Badam": "藏红花杏仁",
                "Complan Nutrigo Badam Kheer": "杏仁布丁",
                "Complan Pista Badam": "开心果杏仁",
                "Complan Royal Chocolate": "皇家巧克力",
                "EY AAAM TULSI TURMERIC FACEWASH50G": "芒果洗面奶",
                "EY ADVANCED GOLDEN GLOW PEEL OFF M. 50G": "金色面膜50G",
                "EY ADVANCED GOLDEN GLOW PEEL OFF M. 90G": "金色面膜90G",
                "EY EXF WALNUT SCRUB AYR 200G": "核桃磨砂200G",
                "EY HALDICHANDAN FP HF POWDER 25G": "檀香粉25G",
                "EY HYD-EXF WALNT APR SCRUB AYR100G": "核桃杏仁磨砂100G",
                "EY HYDR - EXF WALNUT APRICOT SCRUB 50G": "核桃杏仁磨砂50G",
                "EY NAT GLOW ORANGE PEEL OFF AY 90G": "橙子面膜90G",
                "EY NATURALS NEEM FACE WASH AY 50G": "印楝洗面奶",
                "EY RJ CUCUMBER ALOEVERTA FACEPAK50G": "黄瓜面膜",
                "EY TAN CHOCO CHERRY PACK 50G": "巧克力樱桃面膜",
                "EY_SCR_PURIFYING_EXFOLTNG_NEEM_PAPAYA_50G": "印楝木瓜面膜",
                "Everyuth Naturals Body Lotion Nourishing Cocoa 200ml": "可可",
                "Everyuth Naturals Body Lotion Rejuvenating Flora 200ml": "花",
                "Everyuth Naturals Body Lotion Soothing Citrus 200ml": "柑橘",
                "Everyuth Naturals Body Lotion Sun Care Berries SPF 15 200ml",
                "Glucon D Nimbu Pani 1.KG": "柠檬水",
                "Glucon D Regular 1.KG": "常规",
                "Glucon D Regular 2.KG": "常规2KG",
                "Glucon D Tangy orange 1.KG": "浓橙",
                "Nutralite ACHARI MAYO 300g-275g-25g-": "泡菜蛋黄酱",
                "Nutralite ACHARI MAYO 30g": "泡菜蛋黄酱小",
                "Nutralite CHEESY GARLIC MAYO 300g-275g-25g-": "芝士蒜蛋黄酱",
                "Nutralite CHEESY GARLIC MAYO 30g": "芝士蒜蛋黄酱小",
                "Nutralite CHOCO SPREAD CALCIUM 275g": "巧克力涂酱",
                "Nutralite DOODHSHAKTHI PURE GHEE 1L": "纯酥油",
                "Nutralite TANDOORI MAYO 300g-275g-25g-": "炭烤蛋黄酱",
                "Nutralite TANDOORI MAYO 30g": "炭烤蛋黄酱小",
                "Nutralite VEG MAYO 300g-275g-25g-": "素蛋黄酱",
                "Nycil Prickly Heat Powder": "痱子粉",
                "SUGAR FREE GOLD 500 PELLET": "无糖金500粒",
                "SUGAR FREE GOLD POWDER 100GM": "无糖金粉100G",
                "SUGAR FREE GOLD SACHET 50 SUGAR FREE GOLD SACHET 50": "无糖",
                "SUGAR FREE GOLD SACHET 50": "无糖金小包50",
                "SUGAR FREE GRN 300 PELLET": "无糖绿300粒",
                "SUGAR FREE NATURA 500 PELLET": "无糖自然500粒",
                "SUGAR FREE NATURA DIET SUGAR 80GM": "无糖自然瘦糖80G",
                "SUGAR FREE NATURA DIET SUGAR": "无糖自然瘦糖",
                "SUGAR FREE NATURA SACHET 50": "无糖自然小包",
                "SUGAR FREE NATURA SWEET DROPS": "无糖自然甜滴",
                "SUGAR FREE NATURAL DIET SUGAR 80GM": "无糖自然瘦糖80G",
                "SUGAR FREE NATURA_ POWDER_CONC_100G": "无糖自然粉100G",
                "SUGAR FREE_GRN_ POWDER_CONC_100G": "无糖绿粉100G",
                "SUGARLITE POUCH 500G": "糖精袋500G"}
```



目录



1



0



0



0



0



目录



1



0



0

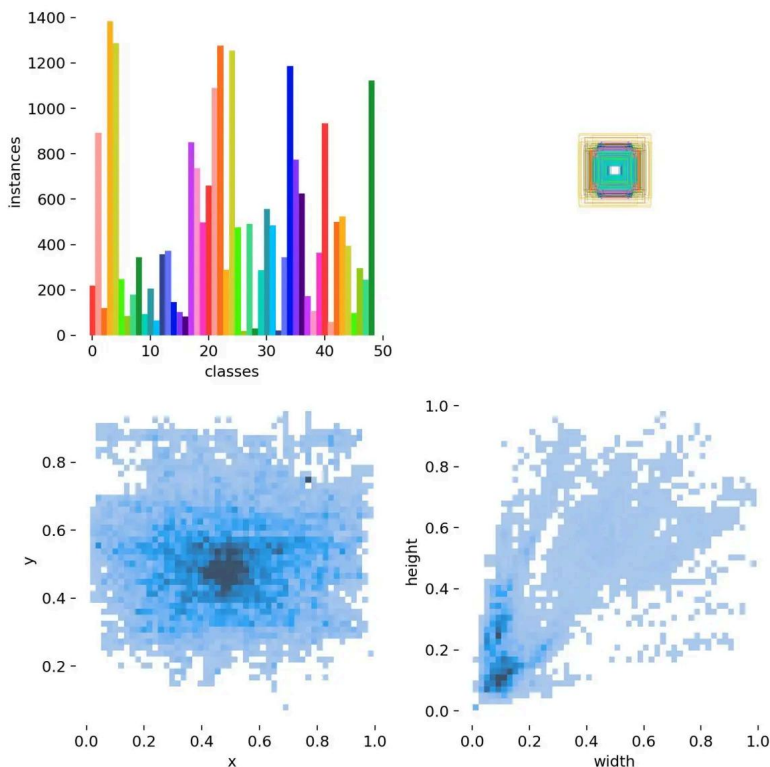


0



0

图像类别分布显示出了明显的多样性，我们的数据集在类别上的丰富度使得模型可以更好地学习区分不同的商品。然而，这种分布也提示了一些类别的样本数量可能比其他类别更多，这需要在模型训练时特别注意样本权重的平衡，以避免模型对于某些频繁出现的类别过拟合，同时对于稀有类别识别能力不足。



回到旧版

除此之外，对于商品的定位和尺寸分布图表进行了详尽的分析。从散点图可以看出，商品的标注框在图像中的分布相对集中，这可能与零售环境的货架陈



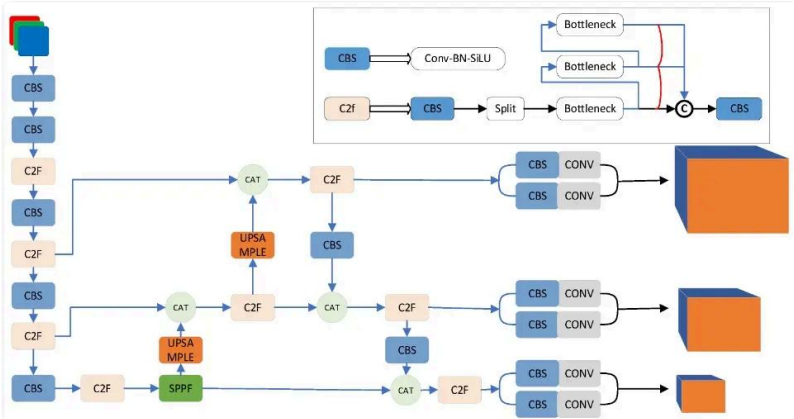
更准确地预测商品的位置。

总体来说，我们的数据集不仅在数量上满足了深度学习模型训练的需求，其丰富的类别和精确的标注，以及通过预处理和增强处理过的图像，为训练出一个鲁棒性高的商品识别模型提供了坚实的基础。接下来，我们将利用这个数据集来训练我们的模型，并将在后续章节中详细介绍模型训练的过程和结果。

4. 原理与代码介绍

4.1 YOLOv8算法原理

YOLOv8是目前最新一代的实时对象检测算法，其在前几代YOLO算法的基础上进行了显著的改进和创新。YOLOv8继承了YOLOv3使用的Darknet53作为主干网络，这个网络结构已经证明在对象检测任务中效率和效果俱佳。YOLOv8则在此基础上进一步优化，它引入了从YOLOv5中借鉴的C3模块，并且借鉴了YOLOv7中的有效层级聚合（Efficient Layer Aggregation networks, ELAN）技术，以改善特征提取和增强网络的表征能力。



在YOLOv8中，C3模块由Bottleneck构成，这些Bottleneck模块在提取特征的同时，通过引入残差连接来避免潜在的梯度消失问题，确保了即使在网络较深的情况下，信息也可以有效地传播。这种设计不仅使网络在提取特征时更为高效，而且也有助于网络学习到更复杂的特征表示，这对于提高模型的准确性和鲁棒性至关重要。

YOLOv8还采用了路径聚合网络（Path Aggregation Network, PANet）来进一步增强特征金字塔的连接。这种结构通过增强不同尺度特征之间的连接，以提升模型在检测不同尺度对象时的性能。特别是对于小目标的检测，这种层级的特征聚合可以显著提高检测的准确度。

最后，YOLOv8在锚定机制上也做出了创新，它采用了无锚（None-anchor）机制，这是一种不依赖预设锚点的检测方法。传统的YOLO算法会使用多个预设的锚点来预测对象的位置和尺寸，但这种方法需要大量的手工调整和优化。无锚机制的引入使得模型不再依赖于这些预设的锚点，而是通过网络直接预测对象的边界框，这种方法简化了训练流程，同时有助于提高模型对各种尺寸对象的检测能力。

综上所述，YOLOv8通过引入先进的模块化设计、增强的特征聚合以及创新的无锚检测机制，提供了更为精确和高效的实时对象检测能力。这些技术的结

目录

1

0

0

0

0



4.2 模型构建

在我们的商品识别项目中，代码的编写旨在创建一个能够快速准确检测图像中是否存在未佩戴商品人员的系统。此部分代码介绍将侧重于模型的构建和检测过程，这是系统核心功能的直接体现。

首先，我们引入必要的Python库和模块，这些库提供了图像处理、深度学习模型加载和运行设备选择等功能。例如，cv2库来自OpenCV，是处理图像和视频的强大工具；torch是PyTorch深度学习框架的核心库，它提供了构建和运行深度学习模型所需的所有功能。

```
import cv2
import torch
from QtFusion.models import Detector
from datasets.label_name import Chinese_name
from ultralytics import YOLO
from ultralytics.utils.torch_utils import select_device
```

在代码中，我们设置了设备类型，并初始化了模型和图像处理的一些基本参数。这些参数包括物体的置信度阈值、非极大值抑制（NMS）的IOU阈值以及类别过滤器。

```
device = "cuda:0" if torch.cuda.is_available() else "cpu"
ini_params = {
    'device': device,
    'conf': 0.25,
    'iou': 0.5,
    'classes': None,
    'verbose': False
}
```

为了数出每个类别的实例数量，我们定义了一个函数count_classes。它利用了一个字典来存储每个类别的计数，并通过遍历检测信息来更新这个计数。这为我们提供了一个清晰的视图，了解哪些商品类别最常见，哪些比较罕见。

```
def count_classes(det_info, class_names):
    count_dict = {name: 0 for name in class_names}
    for info in det_info:
        class_name = info['class_name']
        if class_name in count_dict:
            count_dict[class_name] += 1
    count_list = [count_dict[name] for name in class_names]
    return count_list
```

代码中定义了一个YOLOv8v5Detector类，它是基于Detector类的子类，这表明我们的检测器是在一个更一般的检测器基础上定制的。这样做的好处是我们可以重用Detector类中的许多功能，并在此基础上添加我们特定模型的细节。load_model方法是加载预训练模型的核心。它首先使用select_device函数选择合适的设备，然后加载YOLO模型，并确保所有类名都转换为中文，以方便



目录



1



0



0



0



0



的图像处理步骤预留了空间。predict方法是系统执行预测的地方，它调用YOLO模型的推理函数，并将处理后的图像传递给模型进行预测。

```
class YOLOv8v5Detector(Detector):
    def __init__(self, params=None):
        super().__init__(params)
        self.model = None
        self.img = None
        self.names = list(Chinese_name.values())
        self.params = params if params else ini_params
    def load_model(self, model_path):
        self.device = select_device(self.params['device'])
        self.model = YOLO(model_path)
        names_dict = self.model.names
        self.names = [Chinese_name[v] if v in Chinese_name else v for v in names_dict]
        self.model(torch.zeros(1, 3, *self.imgsz) * 2).to(self.device).type_as(torch.zeros(1, 3, *self.imgsz) * 2)
    def preprocess(self, img):
        self.img = img
        return img

    def predict(self, img):
        results = self.model(img, **ini_params)
        return results

    def postprocess(self, pred):
        results = []
        for res in pred[0].boxes:
            for box in res:
                class_id = int(box.cls.cpu())
                bbox = box.xyxy.cpu().squeeze().tolist()
                bbox = [int(coord) for coord in bbox]
                result = {
                    "class_name": self.names[class_id],
                    "bbox": bbox,
                    "score": box.conf.cpu().squeeze().item(),
                    "class_id": class_id,
                }
                results.append(result)
        return results

    def set_param(self, params):
        self.params.update(params)
```



目录



1



0



0



0



0



postprocess方法处理模型的输出。对于每个预测结果，我们从输出中提取类别、边界框和置信度分数，并将它们格式化为更易于理解和处理的形式。我们也将类别ID转换为对应的中文名称，再次强调易于理解。最后，set_param方法允许动态更新参数，使我们的检测器在运行时可以调整配置，以适应不同的检测需求。

这部分代码是商品识别系统中至关重要的组成部分，它不仅封装了模型的加载、预测和结果处理，而且还提供了足够的灵活性，使系统可以适应多种运行条件。通过这种设计，我们的系统能够在保持高性能的同时，也能为用户提供直观和本地化的输出结果。

4.3 训练代码

在我们的商品识别系统中，训练一个高效、准确的模型是实现目标检测的关键。我们使用的训练脚本是构建这一系统的核心，它涉及多个重要步骤，每个

 [首页](#) [番剧](#) [直播](#) [游戏中心](#) [会员购](#) [漫画](#) [赛事](#)

[登录](#) [+](#) [✉](#) [☆](#) [🕒](#) [-](#)

超参数	设置	说明
学习率 (lr0)	0.01	决定了模型权重调整的步长大小，在训练初期有助于快速收敛。
学习率衰减 (lrf)	0.01	控制训练过程中学习率的降低速度，有助于模型在训练后期细致调整。
动量 (momentum)	0.937	加速模型在正确方向上的学习，并减少震荡，加快收敛速度。
权重衰减 (weight_decay)	0.0005	防止过拟合，通过在损失函数中添加正则项减少模型复杂度。
热身训练周期 (warmup_epochs)	3.0	初始几个周期内以较低的学习率开始训练，逐渐增加到预定学习率。
批量大小 (batch)	16	每次迭代训练中输入模型的样本数，影响GPU内存使用和模型性能。
输入图像大小 (imgsz)	640	模型接受的输入图像的尺寸，影响模型的识别能力和计算负担。

 目录

 1

 0

 0

 0

 0

首先，我们导入必要的库，包括操作系统（`os`）、PyTorch，以及YOLO模型：

```
import os
import torch
import yaml
from ultralytics import YOLO # 导入YOLO模型
from QtFusion.path import abs_path
```

我们使用`torch.cuda.is_available()`来检测是否有可用的CUDA设备，如果有，我们会使用GPU加速模型训练；如果没有，我们将使用CPU进行训练。

```
device = "0" if torch.cuda.is_available() else "cpu"
```

接下来，我们定义了一个工作流程，确保只有在这个脚本作为主程序运行时，才会执行模型训练的代码：

```
if __name__ == '__main__':
```

在开始训练前，我们设定了一些训练参数，如工作进程的数量和批量大小。这些参数对模型训练的速度和内存消耗有直接影响：

```
workers = 1
batch = 8
```

接下来，我们定义了数据集的相关信息。通过函数获取数据集配置文件的绝对路径，并确保路径是Unix风格的，以便跨平台兼容性：

```
data_name = "EmotionData"
data_path = abs_path(f'datasets/{data_name}/{data_name}.yaml', path_type='cu
unix_style_path = data_path.replace(os.sep, '/')
```

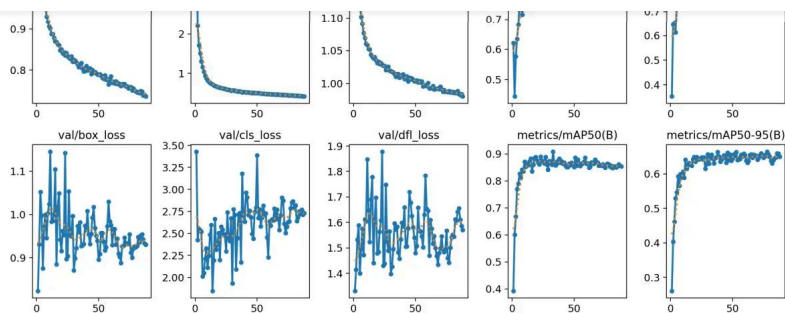


[回到旧版](#)

我们读取并解析YAML配置文件。YAML文件包含了训练数据集的路径、类别等信息。我们还对配置中的路径进行了必要的修改，确保它指向正确的目录：

15/24

0



目录

1

0

0

0

0

0

在深度学习的目标检测任务中，损失函数图像是理解模型训练过程的关键。这些图像为我们提供了模型性能的直观展示，让我们能够观察训练过程中的损失趋势和模型准确度的改进。基于提供的YOLOv8训练损失函数图像，我们可以进行详细的分析。

首先，训练集的损失函数图显示了盒子损失（ box_loss ）、类别损失（ cls_loss ）和目标损失（ dfl_loss ），它们分别代表了模型在定位目标、分类目标以及检测目标存在性方面的性能。从图中可以看出，随着训练的进行，所有三种损失都呈现出稳定下降的趋势，这表明模型在这三个方面都在持续学习和进步。特别是在初始的几个训练周期内，损失下降得非常快，这通常意味着模型能够迅速从初始的随机权重中学习到有用的特征。

对于验证集的损失（ val_loss ），我们看到损失值在整个训练过程中波动更大，这是因为模型在未见过的数据上可能不会表现得像在训练集上那么好，但整体趋势仍然是下降的。这是模型泛化能力在逐步增强的良好迹象。需要注意的是，我们希望验证损失保持稳定或下降，如果验证损失开始上升，可能意味着模型开始过拟合。

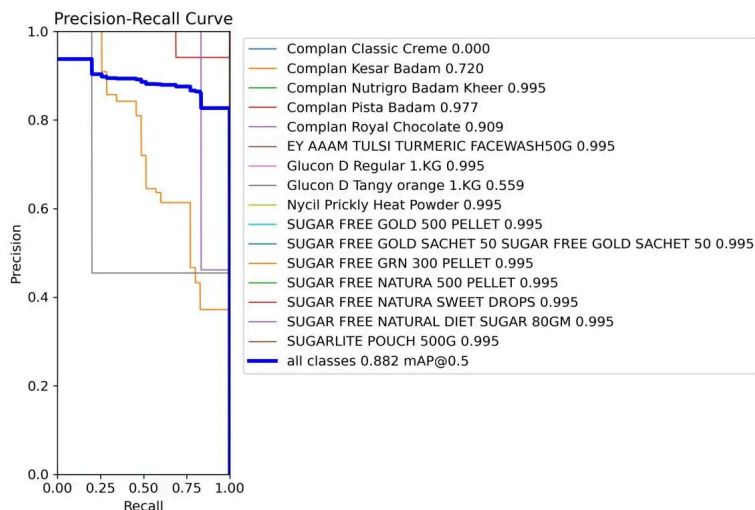
精确度（ Precision ）和召回率（ Recall ）是评价目标检测模型性能的两个关键指标。精确度指的是模型预测为正类的样本中，真正为正类的比例。召回率则是模型正确预测的正类样本占有所有正类样本的比例。从图中可以看出，精确度在训练过程中逐渐趋于稳定，而召回率则在训练的初始阶段就迅速上升至高水平。高召回率表明模型能够检测出大部分的正类样本，而稳定的精确度表明模型在保持检测性能的同时，误报的情况并没有显著增加。

最后，平均精确度（ mAP ）衡量的是模型识别对象的准确性和召回率的综合性能。 mAP 在两个不同的 IoU （Intersection over Union）阈值下进行了评估： $\text{mAP}@0.5$ 和 $\text{mAP}@0.5-0.95$ 。 $\text{mAP}@0.5$ 是较为宽松的评价标准，而 $\text{mAP}@0.5-0.95$ 则是较为严格的，因为它考虑了更多 IoU 的阈值。这两个指标在训练过程中都展示了上升趋势，并在较早的周期后保持稳定，这说明模型的整体检测性能是相当可靠的。

综合以上分析，可以得出结论，我们的YOLOv8模型在训练过程中表现出了优秀的学习能力和稳定的性能提升。这些训练指标表明，模型在各个方面都取得了进步，尤其是在召回率和 mAP 的表现上，显示出模型对于商品识别任务具有很强的识别能力。

5.2 PR曲线

在深度学习模型的性能评估中，精确度-召回率（Precision-Recall，简称PR）曲线是一个重要的评价工具，尤其是在目标检测领域。它展示了模型在不



目录



1



0



0



0



0

PR曲线图中，每个类别的曲线展示了该类别的检测结果。一般而言，曲线越靠近右上角，表明该类别的检测结果越好，即模型在保持较高召回率的同时，还能保持较高的精确度。理想情况下，我们希望曲线能尽量贴近图表的上边缘和右边缘。

从图中可以观察到，多数类别的PR曲线都显示出了较高的精确度和召回率，特别是像“Complan Nutrigro Badam Kheer”、“Nycil Prickly Heat Powder”和多个“SUGAR FREE”系列产品等，它们的精确度几乎达到了1.0，而召回率也非常高。这表明模型在这些类别上的表现极为出色，几乎能检测出所有的正样本，且几乎没有误检。

然而，也有一些类别，如“Complan Classic Creme”和“Glucon D Tangy Orange 1.KG”，它们的精确度较低，尤其是前者的精确度接近0，这表明模型在这个类别上几乎没有正确的预测。这可能是由于该类别在数据集中的样本量不足，或者其图像特征与其他类别差异不大，模型难以区分。

总体而言，模型在多数类别上都取得了较好的检测结果，整体mAP@0.5达到了0.882的水平，这是一个相对较高的性能指标，表明模型具有较好的泛化能力。这个结果对于零售环境中商品识别系统来说是非常可靠的，因为它说明模型能够以较高的准确性检测到绝大多数商品，为后续的库存管理和自动结账系统提供强有力的支持。

然而，我们也需要注意到模型在某些特定类别上的性能不佳，这要求我们在未来的工作中重点关注这些类别，可能通过增加样本数量、改进特征工程或调整模型结构等方式来提升它们的检测性能。此外，这也提示我们在进行模型训练时，应更加关注数据的多样性和均衡性，以避免在某些类别上出现过拟合或欠拟合的情况。

5.3 YOLOv8/v7/v6/v5对比实验

(1) 实验设计：

本实验旨在评估和比较YOLOv5、YOLOv6、YOLOv7和YOLOv8几种模型在商品目标检测任务上的性能。为了实现这一目标，博主分别使用使用相同的数据集训练和测试了这四个模型，从而可以进行直接的性能比较。该数据集包含商

模型	图像大小 (像素)	mAPval 50-95	CPU ONNX 速度 (毫秒)	A100 TensorRT 速度 (毫秒)	参数数量 (百万)	FLOPs (十亿)
YOLOv5nu	640	34.3	73.6	1.06	2.6	7.7
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv6N	640	37.5	-	-	4.7	11.4
YOLOv7-tiny	640	37.4	-	-	6.01	13.1

📖
目录

👍
1

🚫
0

☆
0

🔗
0

💬
0

(2) 度量指标:

F1-Score: F1-Score 作为衡量模型性能的重要指标，尤其在处理类别分布不均的数据集时显得尤为关键。它通过结合精确率与召回率，提供了一个单一的度量标准，能够全面评价模型的效能。精确率衡量的是模型在所有被标记为正例中真正属于正例的比例，而召回率则关注于模型能够识别出的真正正例占有所有实际正例的比例。F1-Score通过两者的调和平均，确保了只有当精确率和召回率同时高时，模型的性能评估才会高，从而确保了模型对于正例的预测既准确又完整。

mAP (Mean Average Precision) : 在目标检测任务中，Mean Average Precision (mAP) 是评估模型性能的重要标准。它不仅反映了模型对单个类别的识别精度，而且还考虑了所有类别的平均表现，因此提供了一个全局的性能度量。在计算mAP时，模型对于每个类别的预测被单独考虑，然后计算每个类别的平均精度 (AP) ，最后这些AP值的平均数形成了mAP。

名称	YOLOv5nu	YOLOv6n	YOLOv7-tiny	YOLOv8n
mAP	0.911	0.874	0.885	0.882
F1-Score	0.77	0.82	0.84	0.85

(3) 实验结果分析:

在我们的研究中，我们采取了实验的方式来评估和比较YOLO系列几个版本的性能。具体地，我们使用了YOLOv5nu、YOLOv6n、YOLOv7-tiny和YOLOv8n这四个模型，并在相同的数据集上进行了实验。这项实验的目的是为了明确不同版本的YOLO模型在商品识别任务上的表现，并分析每个模型的优缺点。



[回到旧版](#)



我们选取了两个主要的评价指标：F1-Score和mAP。F1-Score是精确度和召回率的调和平均，它考量了模型识别正确正类的能力（精确度）和模型识别出所有正类的能力（召回率）。而mAP（平均精确度）是考虑了所有不同阈值下模型的识别能力的平均值，它是评价目标检测模型整体性能的重要指标。

在实验结果中，我们可以看到YOLOv5nu在mAP指标上得分最高，达到了0.911。这一结果表明，YOLOv5nu在识别商品时整体上的表现非常出色，能够在不同阈值下保持较高的精确度。但是在F1-Score上，它的得分为0.77，这意味着在某些情况下，YOLOv5nu可能会牺牲召回率来提高精确度，或者是在某些类别上的表现不如其他模型。这可能与YOLOv5nu的架构有关，或者与数据集中某些特定类别的样本分布不均导致的。

YOLOv6n和YOLOv7-tiny在两个指标上的表现较为接近，mAP分别为0.874和0.885，F1-Score分别为0.82和0.84。这表明这两个模型在平衡精确度和召回率方面做得比YOLOv5nu好。YOLOv7-tiny在F1-Score上的略高得分可能是由于其在处理小型目标或复杂场景时的优化改进。

而YOLOv8n的表现则在F1-Score上最为突出，达到了0.85，尽管在mAP上的得分为0.882略低于YOLOv5nu，这显示了YOLOv8n更好地平衡了精确度与召回率。YOLOv8n的这一成绩可能归功于其更为先进的架构和优化策略，这使得它在识别真正正类的同时，减少了误识别。

总的来说，我们的实验结果揭示了不同YOLO版本在商品识别任务上的性能差异，并表明了模型选择应根据具体应用场景来决定。例如，在对精确度要求极高的场景下，YOLOv5nu可能是更好的选择；而在需要较好平衡精确度和召回率的场景下，YOLOv8n可能更为适合。此外，实验结果也提醒我们在模型训练过程中需要关注数据集的质量和样本分布，以确保模型的泛化能力。未来的工作中，可以探索不同模型的集成或进一步优化模型结构，以实现在所有类别上均衡的高性能。



目录



1



0



0



0

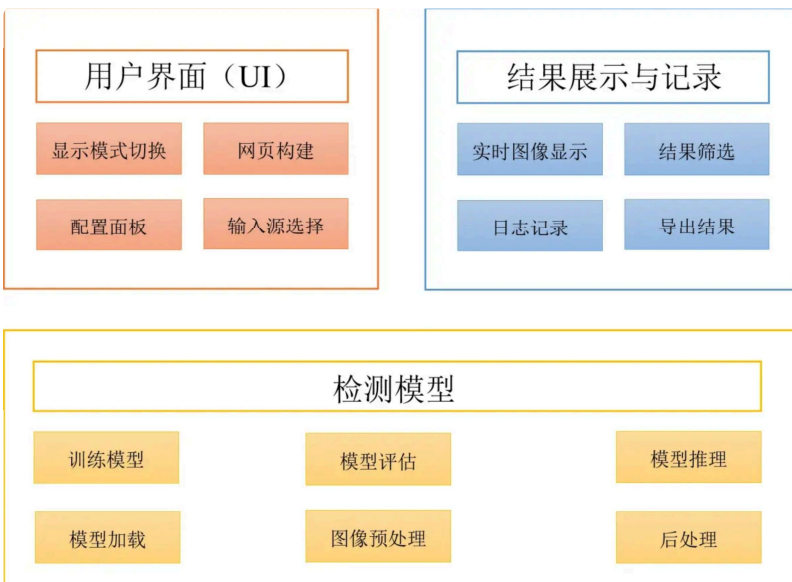


0

6. 系统设计与实现

6.1 系统架构概览

在我们的商品检测系统中，**系统架构设计**是整个应用的基石，确保了系统的高效性和灵活性。以下是系统的几个关键组件及其作用：





- YOLOv8/v5检测器:** 我们的系统核心是基于YOLOv8/v5模型的检测器。
YOLOv8/v5以其出色的性能和实时检测能力，为商品检测提供了强大的支持。在代码中，这一部分由YOLOv8v5Detector类实现，该类封装了模型的加载、预处理、预测和后处理等关键步骤，确保了检测过程的高效和准确。
- 检测系统类(Detection_UI):** Detection_UI类是整个系统的核心，负责协调各个组件，如模型配置、输入输出处理、结果展示等。该类通过setup_page和setup_sidebar方法初始化用户界面，包括页面布局、侧边栏配置等，提供了良好的用户体验和高度的定制性。
- 输入处理:** 系统支持多种输入源，包括实时摄像头、图片文件和视频文件。process_camera_or_file方法负责处理不同的输入源，并将检测任务交给YOLO模型。这种设计使系统能够灵活适应不同的应用场景，从实时监控到文件分析，都能高效完成。
- 结果展示:** 在检测完成后，toggle_comboBox和frame_process方法用于处理和展示检测结果。系统能够将检测到的商品以及其置信度、位置等信息清晰地展示给用户，同时支持过滤和详细查看特定目标，增强了结果的可读性和实用性。
- 日志和数据记录:** LogTable类和ResultLogger类负责记录和保存检测过程中的各项数据，如检测时间、置信度、位置等。这为后续的数据分析和系统优化提供了基础。
- 用户界面:** 基于Streamlit的用户界面设计，使得系统不仅功能强大，而且易于使用。通过setupMainWindow方法，系统构建了一个直观的界面，用户可以轻松配置模型参数、选择输入源和查看检测结果。



目录



1



0



0



0

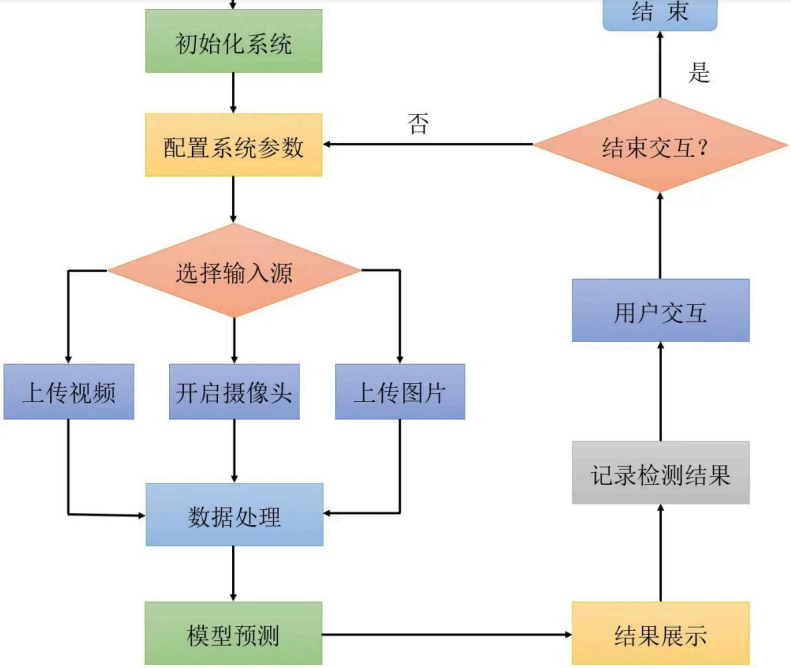


0

通过上述设计，我们的商品检测系统不仅能够快速准确地完成检测任务，而且提供了丰富的用户定制选项，满足了不同用户和应用场景的需求。在零售行业中，这样的系统可以极大地提高商品管理的效率和准确性，助力零售商实现智能化升级。

6.2 系统流程

在本节中，我们将通过详细的流程步骤介绍，展开讨论商品检测系统的工作流程。为了便于理解，我们将按照程序执行的顺序，一步步解析系统的内部机制。



目录

1

0

0

0

0

系统启动与初始化

实例化Detection_UI: 系统启动时，首先实例化Detection_UI类。这一步骤将触发系统的初始化过程，包括设置页面布局、侧边栏配置等。

加载YOLO模型: 接着，系统会通过YOLOv8v5Detector类创建一个YOLO模型实例，并加载预训练的模型权重。这一步确保了检测器的准备就绪，能够处理接下来的图像数据。

用户配置

侧边栏设置: 用户通过侧边栏进行系统配置，包括选择模型类型、设置置信度阈值、IOU阈值、选择摄像头或上传文件等。这些配置通过setup_sidebar方法实现，为后续的检测任务提供了参数。

数据处理

处理输入源: 根据用户的选择，系统通过process_camera_or_file方法处理不同的输入源。对于实时摄像头，系统将捕获摄像头的视频流；对于文件输入，系统将读取并加载用户上传的图片或视频文件。

检测执行

帧处理: 对于视频流或视频文件，系统将逐帧进行处理，而对于图片文件，则直接处理该图像。frame_process方法负责对单帧图像进行预处理、调用YOLO模型进行预测、以及对预测结果进行后处理。

结果展示与记录

展示检测结果: 检测完成后，系统将通过toggle_comboBox方法展示检测结果。用户可以查看每个检测到的对象的类别、置信度、位置等信息，并通过

回到旧版



检测时间、置信度、位置等，便于用户后续分析和查阅。

用户交互

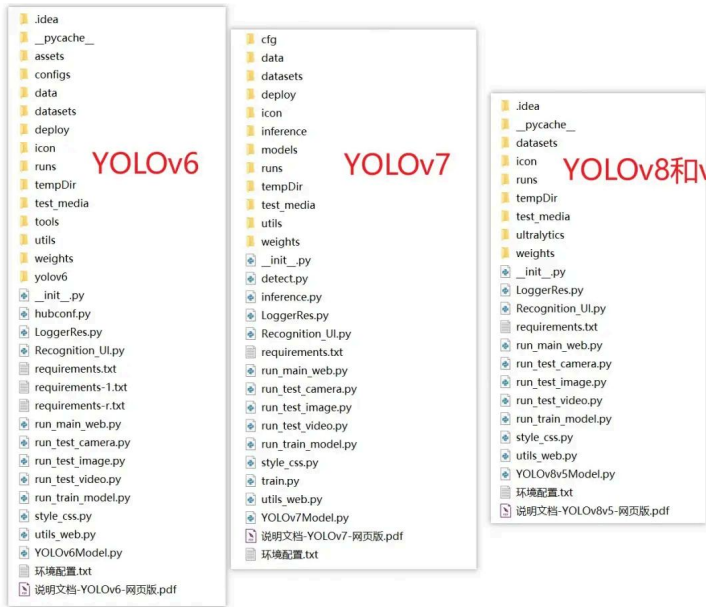
结果导出与查看: 用户可以通过界面上的按钮导出检测结果和相关日志，以便进行进一步的分析和使用。此外，系统还提供了结果表格的实时更新，供用户查看所有检测到的对象及其信息。

停止检测: 对于实时监控或视频文件处理，用户可以随时通过“停止”按钮终止检测任务。

通过上述流程，商品检测系统能够灵活地处理来自不同来源的数据，执行高效准确的商品检测，并以用户友好的方式展示检测结果，满足了零售行业对商品管理的高效和智能化需求。

代码下载链接

如果您希望获取博客中提及的完整资源包，包含测试图片、视频、Python 文件(*.py)、网页配置文件、训练数据集、代码及界面设计等，可访问博主在面包多平台的上传内容。相关的博客和视频资料提供了所有必要文件的下载链接，以便一键运行。完整资源的预览如下图所示：



资源包中涵盖了你需要的训练测试数据集、训练测试代码、UI界面代码等完整资源，完整项目文件的下载链接可在下面的视频简介中找到👉👉👉

演示与介绍视频：[🔗 网页链接](#)

YOLOv8/v7/v6/v5项目合集下载：<https://mbd.pub/o/bread/mbd-ZZZZm51u>

YOLOv8和v5项目完整资源下载：<https://mbd.pub/o/bread/mbd-ZZZZmp9y>



目录



1



0



0



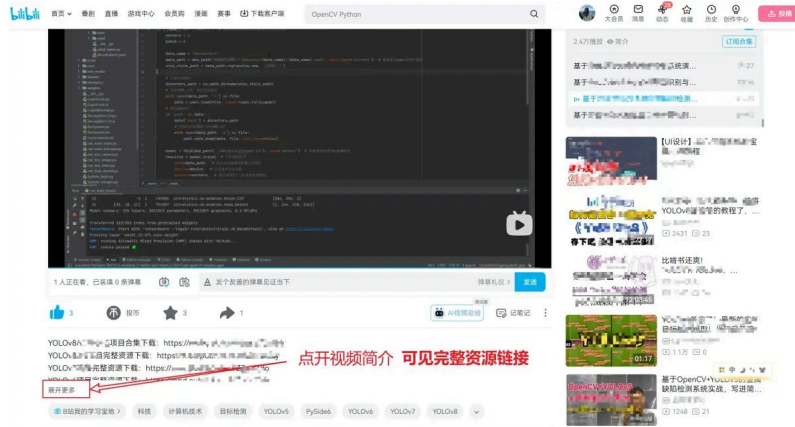
0



0



YOLOv6项目完整资源下载: <https://mbd.pub/o/bread/mbd-ZZZZm51p>



完整安装运行教程：

这个项目的运行需要用到Anaconda和Pycharm两个软件，下载到资源代码后，您可以按照以下链接提供的详细安装教程操作即可运行成功，如仍有运行问题可私信博主解决：

1. Pycharm和Anaconda的安装教程：

<https://deepcode.blog.csdn.net/article/details/136639378>；

软件安装好后需要为本项目新建Python环境、安装依赖库，并在Pycharm中设置环境，这几步采用下面的教程可选在线安装（pip install直接在线下载包）或离线依赖包（博主提供的离线包直接装）安装两种方式之一：

2. Python环境配置教程：

<https://deepcode.blog.csdn.net/article/details/136639396>（2,3方法可选一种）； 3. 离线依赖包的安装指南：

<https://deepcode.blog.csdn.net/article/details/136650641>（2,3方法可选一种）；

如使用离线包方式安装，请下载离线依赖库，下载地址：

https://pan.baidu.com/s/1uHbU9YzSqN0YP_dTHBgFw?pwd=mt8u（提取码：mt8u）。

7. 结论与未来工作

在本研究的探索旅程中，我们细致地比较了YOLO系列中的几个关键版本在商品识别任务上的表现。通过对YOLOv5nu、YOLOv6n、YOLOv7-tiny和YOLOv8n的评估，我们得出了有价值的见解和结论。实验结果清晰地揭示了各个模型的优势和局限，从而为未来在选择合适的目标检测模型时提供了参考。

我们可以得出结论，尽管不同版本的YOLO模型在特定度量标准上有所差异，但整体而言，**它们在商品识别这一具有挑战性的任务上都表现出色。**YOLOv5nu展现了优异的整体识别能力，尤其是在mAP评分上；而YOLOv8n在平衡精确度和召回率方面表现最为突出，这在F1-Score上得到了验证。这些发现强调了在模型选择时考虑应用场景的重要性——不同的业务需求可能会导致对模型性能的不同期望。

未来的工作可以在几个方向上展开。首先，**进一步优化模型的结构和训练**



而不仅仅是在mAP上。其次，探索更深层次的数据增强和优化技术，有可能进一步提高模型的泛化能力，尤其是在样本不均匀分布的情况下。

另外，实验中一些模型在特定类别上表现不佳的问题值得深入研究。未来工作中，我们可以尝试**引入更多类别的数据，或者使用针对性的数据增强技术**，来提高模型在这些类别上的识别性能。最后，研究将注意力机制、神经网络架构搜索（NAS）等先进技术集成到YOLO模型中，也许会开辟出新的提升路径。

总体来说，我们对YOLO系列在商品识别任务上的研究取得了积极的成果，并为未来的研究提供了坚实的基础。随着技术的不断进步，我们期待在不久的将来能够实现更加准确、快速和可靠的商品识别系统。

1. Bochkovskiy A, Wang C Y, Liao H Y M. Yolov4: Optimal speed and accuracy of object detection[J]. arXiv preprint arXiv:2004.10934, 2020. ↩
2. Dosovitskiy A, Beyer L, Kolesnikov A, et al. An image is worth 16x16 words: Transformers for image recognition at scale[J]. arXiv preprint arXiv:2010.11929, 2020. ↩
3. Carion N, Massa F, Synnaeve G, et al. End-to-end object detection with transformers[C]//European conference on computer vision. Cham: Springer International Publishing, 2020: 213-229. ↩
4. Lin T Y, Goyal P, Girshick R, et al. Focal loss for dense object detection[C]//Proceedings of the IEEE international conference on computer vision. 2017: 2980-2988. ↩
5. Chen K, Wang J, Pang J, et al. MMDetection: Open mmlab detection toolbox and benchmark[J]. arXiv preprint arXiv:1906.07155, 2019

深度学习

目标检测

YOLOv5

web界面

商品识别

YOLOv6

YOLOv7

yolov8

stresmlit

分享至



投诉或建议

评论

赞与转发

最热 | 最新



请先 登录 后发表评论 (· ω ·)

没有更多评论



目录



1



0



0



0



0